



Figure 1: Docker architecture (<https://docs.docker.com/engine/article/img/architecture.svg/>)

```
<code> shell>
$ sudo gpasswd -a $(USER) docker
```

There are three major terminologies to understand when working with Docker.

1. **Dockerfile:** This is a text document, which contains a set of commands that are required to build an image.
2. **Image:** This is a compressed archive that is built from executing the commands in your Dockerfile.
3. **Container:** This is a running instance of an image. You can create any number of containers from an image.

Before we start, let's set up our environment. We will be doing all the work inside `~/dock`.

```
<code> shell>
$ mkdir -p ~/dock/mysql
$ cd ~/dock
# demo rails application
$ git clone https://gitlab.com/technianiack/rails-mrd-demo.git
~/dock/rails-mrd-demo
```

One of the major benefits of using Docker is that most of the popular open source tools are Dockerised by their maintainers and are available for anybody to use. To know what a Dockerfile is and what it does, we will use a MySQL image as a starting point and build our own image locally.

```
<code> shell>
$ cd ~/dock/mysql
$ touch ~/dock/Dockerfile
```

Open the Dockerfile in the editor of your choice and copy the following code inside it:

```
<code> Dockerfile>
#Use official mysql as a starting point
FROM mysql
#Set password for root user
ENV MYSQL_ROOT_PASSWORD=root
```

Now, we are all set to build our first image, which we will name `'mysql_local'`. To build an image from Dockerfile, we have to use 'docker build' as shown below:

```
<code> shell>
$ docker build -t mysql_local .
Sending build context to Docker daemon 2.048 kB
Step 1: FROM mysql
latest: Pulling from library/mysql

386a066cd84a: Pull complete
827c8d62b332: Pull complete
de135f87677c: Pull complete
05822f26ca6e: Pull complete
63ddbdf6165: Pull complete
15fe0fbc587e: Pull complete
93e74acdb291: Pull complete
11c2df82e984: Pull complete
d42a9e6a85c8: Pull complete
e7fb2f3afd87: Pull complete
30724006a583: Pull complete
Digest: sha256:89cc6ff6a7ac9916c3384e864fb04b8ee9415b572f872a
2a4cf5b999dbca81b
Status: Downloaded newer image for mysql:latest
--> d9124e6c552f
Successfully built d9124e6c552f
```

You will see a bunch of 'Pull complete' messages, which is fine because we asked Docker to build 'FROM mysql' in our Dockerfile. Now the MySQL image is stored on your local machine, which means that when you try to build a new image of 'FROM mysql', it won't pull it from the Internet. The very last line says 'Successfully built d9124e6c552f', where the ID is of the newly created image, i.e., `mysql_local`. You can list all the images on your machines with 'docker images' as shown below:

```
$ docker images
REPOSITORY TAG IMAGE ID CREATED SIZE
mysql_local latest d61327195850 1 minutes ago 383.4 MB
\mysql latest d9124e6c552f 4 days ago 383.4 MB
```

As you can see, we now have two images on our machine.

1. **mysql:** The base image.
 2. **mysql_local:** The image we created with the custom root password.
- Let's go ahead and set up a container, and connect to the MySQL prompt just to verify that things are working.

```
<code> shell>
$ docker run -d -p 3306:3306 mysql_local

# Check running containers
$ docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS
```